We Will Know: a reproducible pipeline for a code-animated, word- and beat-synchronised music video, with pre-registered validation of its timing components

Libor Burian (independent researcher)

18 October 2026

Abstract

We describe and evaluate an open pipeline that turns hand-written lyrics into a four-minute music video in which every frame is drawn in code.

Pipeline. One Suno V6 generation, obtained through the third-party sunoapi.org API, yields two sung takes and per-word timings. A rule fixed before generation selects one take. The written lyrics are sequence-aligned to the provider's word timings. Beats are tracked with librosa. A Remotion/Three.js composition then renders ten scenes, which illustrate ambitious interpretability and a scientific theory of deep learning. All visuals are pure functions of the frame number.

What we tested. Each component that decides *when* something happens on screen was tested against a criterion stated in advance.

1. On synthetic click tracks with known beats (12 tempos × 2 noise levels × 2 hop sizes × 5 seeds), the beat tracker at hop 256 placed beats within 15–19 ms (clean) of the truth. It failed the pre-registered criterion at 170 BPM through a half-tempo error. At librosa's default hop 512, the tempo estimates were quantized to errors of up to 3.55 BPM.
2. Against an independent recogniser (faster-whisper), the provider's word onsets agreed to a median of 0.235 s (take 1) and 0.172 s (take 2) over 393 and 401 matched words. Random ±1 s shifts gave 0.59 s and 0.50 s, so the agreement is not an artifact of the metric.
3. A plausible-looking timing-repair heuristic was **falsified** by the pre-registered test: on 43 words it moved onsets a median of 1.14 s *away* from the reference. A minimal post-hoc rule now touches only 2 grossly misplaced words.
4. The final 1920×1080, 30 fps render (7213 frames) matched the song's duration to 0.03 s.

We report these as properties of this pipeline and this song. They are not general claims about the music model.

1. Introduction

Context. In 2026 a genre of short videos appeared in which a language model writes the code for every frame of an animated music video, usually with a song from Suno and React-based rendering through Remotion (e.g. the prompt collection *awesome-opus5-5-videos* on GitHub). Such videos stand or fall on *timing*: lyrics must light up when they are sung, and cuts must land on beats.

The question. Public descriptions of these videos report which tools were used. They do not report whether the timing is right, or how one would know.

The subject of the song. The song, *We Will Know*, is an anthem for an ambitious science of information processing. Its themes:

- the goal of fully reverse-engineering neural networks (leogao, 2025; Goodfire, n.d.);
- the *ladder of abstraction* from neurons and attention heads to behaviors and drives (Goodfire, n.d.);
- the five directions toward a scientific theory of deep learning (Simon et al., 2026);
- an atlas of the mathematics involved;
- "ideal solving": the shortest theory that predicts any behavior and any intervention (Kolmogorov, 1965; Pearl, 2009).

Contributions.

1. An open, single-command, reproducible pipeline: song, word timings, beats, timeline, deterministic render, automated verification. Paid generation is cached and tests run offline.
2. Pre-registered checks of the three timing components (take selection, beat tracking, word timing), each with an explicit falsification criterion, raw data and a control.
3. Two negative results reported in full:
   - a beat-tracker half-tempo failure inside the tested tempo range;
   - a word-timing repair rule that the test showed to be harmful.

   The repair finding changed what ships: without the test, 27 words of the video would have lit up 0.8–2.5 s late.

2. Background and related work

- **Beat tracking.** librosa (McFee et al., 2015) implements dynamic-programming beat tracking (Ellis, 2007). It builds an onset-strength envelope, picks a global tempo from an autocorrelation tempogram with a log-normal prior centred at 120 BPM, then places beats by dynamic programming. Beat accuracy is conventionally scored with an F-measure at a ±70 ms tolerance, the default in `mir_eval` (Raffel et al., 2014). Tempo estimation suffers from *metrical-level* (octave) ambiguity: half or double tempo is a common failure mode.
- **Lyric/word timing.** Whisper (Radford et al., 2023) yields word timestamps through cross-attention alignment. These are known to be imprecise, which motivated forced-alignment add-ons such as WhisperX (Bain et al., 2023). We therefore use Whisper only as an *independent, noisy* reference, never as ground truth.
- **Programmatic video.** Remotion renders React components frame by frame in headless Chrome. A video is a pure function of the frame index, which allows parallel, deterministic rendering. Three.js scenes run via `@remotion/three` with the ANGLE GL backend.
- **Scientific content shown in the video.** Flat minima and generalization (Hochreiter & Schmidhuber, 1997; Keskar et al., 2017); circular features for days and months (Engels et al., 2024); linear space and time representations that are more accurate in larger models (Gurnee & Tegmark, 2023, §3.1); function vectors (Todd et al., 2023); the five research directions of "learning mechanics" (Simon et al., 2026). These are illustrated schematically, and the plots in the video use synthetic data, labelled as such.

3. Method

3.1 Pipeline

A Python wrapper (`uv run anthem`) runs four phases. The phases communicate only through files.

1. **song.**
   - Sends one `POST /api/v1/generate` with custom mode, V6, the hand-written lyrics (2701 characters, 11 sections, 58 lines, 430 words), a style string, negative tags and `duration: 240`.
   - Polls `record-info` until `SUCCESS`.
   - Downloads both takes and covers, and calls `get-timestamped-lyrics` for each take.
   - Caches everything, so no second paid call is ever made implicitly. All requests and responses are logged in full.
2. **timeline.**
   - Parses the lyrics, selects a take (§3.2), aligns the words (§3.3) and tracks beats (§3.4).
   - Starts each section at its first sung word minus 0.1 s, snapped to a beat if one is within ±0.3 s.
   - Writes `timeline.json`: sections and scenes, lines, words, beats, downbeats, and per-frame RMS and onset strength.
3. **render.**
   - Remotion 4.0.536 reads the timeline in `calculateMetadata`, so the video length equals the song length.
   - It stacks a starfield, a spectrum ring, one scene per section, a karaoke overlay (or giant kinetic words in the choruses), beat and cut flashes, and a HUD.
   - Render settings: `--gl=angle`, concurrency 8.
4. **verify.**
   - ffprobe acceptance checks, plus one still per section extracted with ffmpeg.

3.2 Take selection (fixed before generation)

The chosen take maximizes *coverage*, then minimizes `hootCer` (the provider's alignment score; lower is assumed better).

- **Coverage** is the share of written words, after normalization (lowercase, alphanumerics only), matched exactly to a provider word with `success=true`.
- **Why fix it.** Fixing the rule removes the freedom to pick whichever take makes the video look best.

3.3 Word alignment and repair

- **Cleaning and alignment.** Provider tokens arrive with section tags and line breaks glued on (e.g. `"[Verse 1]\nA "`). We strip the tags, split on whitespace, divide each entry's span evenly among its words, and align the result to the written words with `difflib.SequenceMatcher`.
- **Unmatched words.** Equal-length replace blocks are timed but not counted as matches. Deleted words are interpolated between their neighbours.
- **Repair v1** (pre-registered in exp03). A mid-line word longer than 1.2 s had its start moved to 0.5 s before its end. Any mid-line gap over 0.6 s pulled the earlier word up to its successor.
- **Repair v2** (post-hoc, §4.3). Only mid-line words longer than 3.0 s are moved. A preceding word follows only if its successor was moved.

3.4 Beats

- **Tracking.** `librosa.beat.beat_track` on the onset-strength envelope at 22.05 kHz with hop 256, chosen after the first unit test failed at hop 512 (§4.2).
- **Tempo.** Reported as 60 / median inter-beat interval.
- **Downbeats.** Every 4th beat, phase chosen by the summed onset strength. This is a heuristic that assumes 4/4.

3.5 Parameters and their rationale

Parameter	Value	Rationale
Song duration	240 s	~4-minute anthem; the API default of 20 s must be overridden
Video	1920×1080, 30 fps	YouTube 1080p; 33.3 ms frame = the timing resolution the viewer sees
Hop size	256 samples (11.6 ms)	Exp02: beats within 1 video frame; half the tempo-quantization error of 512
Section lead / snap window	0.1 s / ±0.3 s	Cut slightly before the first sung word; never move a cut more than ~half a beat
Repair v2 thresholds	3.0 s; follow if gap > 0.6 s; repaired duration 0.5 s	3.0 s is more than 6 beats at 129 BPM; 0.5 s is about one beat
H1 threshold	0.25 s	About half a beat; set before running, given the expected noise in Whisper's timestamps
Beat F window	±70 ms	`mir_eval` default
Seeds (exp02)	5 per cell	Enough to expose noise sensitivity; the observed between-seed SD was about 0

4. Experiments and results

4.1 Generation and selection (exp01)

- **Generation.** One task took 62 s from submission to `SUCCESS` and returned two takes: 240.40 s and 239.92 s.
- **Alignment entries.** The provider returned 431 and 439 aligned entries, with `success=true` for all of them.
- **Selection.** After cleaning, both takes covered 430/430 written words, so coverage tied at 1.0. The tie-break chose take 1 (hootCer 0.155 vs 0.165). Coverage turned out to be uninformative for this provider, and we do not claim that take 1 is musically better.
- **Section order.** The section boundaries recovered from take 1 follow the written section order.
- **Artifacts.**
  - Take 2 aligned its first line twice.
  - In take 1, "matrix" spanned 7.74 s and began inside the intro.

4.2 Beat tracker vs ground truth (exp02)

Discovery. A unit test written before the implementation (a 120 BPM click track, ±2 BPM) failed with 117.45 BPM. At hop 512 the tempogram's integer lag is 22 frames, and $43.07 \cdot 60 / 22 = 117.45$.

Systematic run. 30-s click tracks at 70–180 BPM. Pre-registered criterion: at hop 256 on clean clicks, every run must have $|\text{error}| \le 2$ BPM and $F \ge 0.95$.

- **Overall: FAIL.** 11 of 12 tempos passed.
- **The failure is a half-tempo lock.** At 170 BPM the tracker reported 85 BPM, with F = 0.661 (precision 1, recall 0.5). The same failure occurred at 180 BPM at hop 512 and on noisy clicks at hop 256.
- **Hop 512 quantization.** Tempo errors reached ±3.55 BPM.
- **The estimator does not remove it.** The median-spacing and tempogram estimators were identical in every cell, both locked to whole-frame periods. This corrects a claim in our own code comments.
- **Post-hoc slope estimator.** A least-squares slope of beat time vs index reduced the error to ≤ 0.01 BPM wherever the metrical level was correct. Since it was found post hoc, the pipeline keeps the pre-specified estimator.
- **Timing precision.** At hop 256, median beat deviation was 15–19 ms (clean) and 23–25 ms (noisy), below one frame.
- **Seed variance.** It was about 0 for tempo and at most ±0.021 for F.
- **The song.** It measures 129.3 BPM (506 beats), inside the validated range. Suno's own style tag says 124 BPM, which argues against a half or double error.

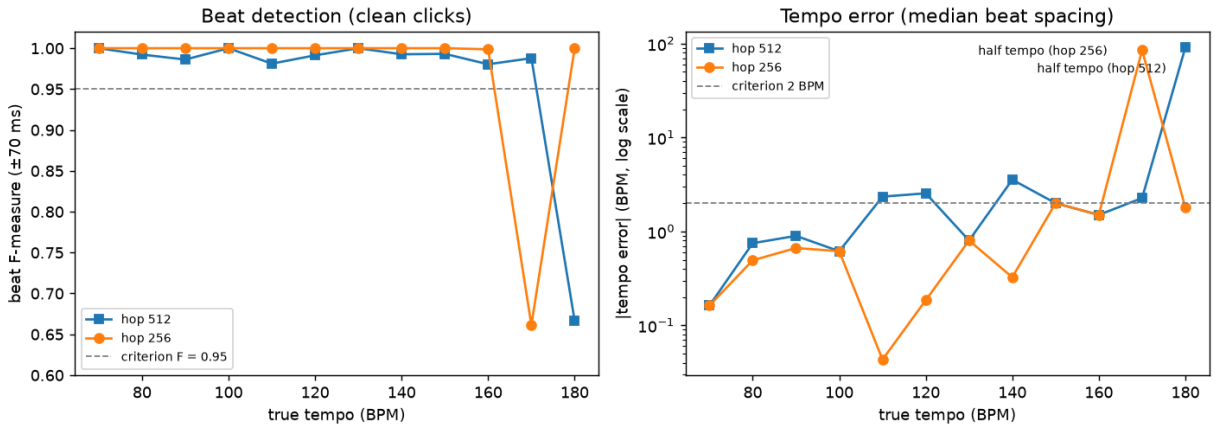


Figure 1: Exp02. Left: beat F-measure; right: absolute tempo error on a log scale, for clean click tracks. Half-tempo failures are labelled.

4.3 Word-onset agreement and the repair (exp03)

Pre-registered design.

- faster-whisper `small.en` (CPU int8) transcribed each take's full mix. It has no access to the lyrics.
- Written and recognized words were matched by text (exact difflib blocks).
- **H1:** median |onset − Whisper onset| ≤ 0.25 s per take.
- **H2:** on repaired words, the repair reduces |Δ| (one-sided Wilcoxon, α = 0.05).
- **Control:** U(−1, 1) s random shifts.

	Take 1 (video)	Take 2
Whisper words / matched pairs	440 / 393	435 / 401
Median |Δ| raw [95% bootstrap CI]	0.235 s [0.225, 0.248]	0.172 s [0.153, 0.190]
Median |Δ| repair v1	0.242 s [0.227, 0.257]	0.179 s [0.158, 0.195]
Median |Δ| repair v2	0.235 s [0.225, 0.248]	0.172 s (no repairs)
Median |Δ| random-shift control (with v2)	0.590 s	0.500 s
Share ≤ 0.25 s / ≤ 0.5 s (v2)	56.2 % / 88.8 %	67.8 % / 90.0 %

H1 passed for both takes.

H2 was falsified for repair v1. Over 43 repaired and matched words (both takes), the median paired change was −1.14 s, i.e. *worse*, with Wilcoxon p ≈ 1.0.

- **Diagnosis.** The 1.2 s threshold flagged real sung notes, e.g. "go" at 1.64 s. Shortening them opened gaps that cascaded the earlier words of those lines 0.8–2.5 s late (27 words in take 1). Whisper and the raw timings agreed on the true positions.
- **v2.** It touches only "A" and "matrix" in take 1, and moves both toward the reference: |Δ| 6.33 → 1.07 s and 6.81 → 0.43 s.
- **v2 is post-hoc and descriptive only.** The intended held-out take (take 2) contains no word that v2 would repair, so no confirmatory test was possible.

Post-hoc analysis. The signed differences show the disagreement is mostly a constant lag: Suno is later than Whisper for 98.5 % (take 1) and 93.8 % (take 2) of words, by a median of 0.234 s and 0.165 s. Around that lag, the spread is only 0.075 s and 0.089 s. Which system is biased cannot be decided without manual annotation, so no offset correction was applied.

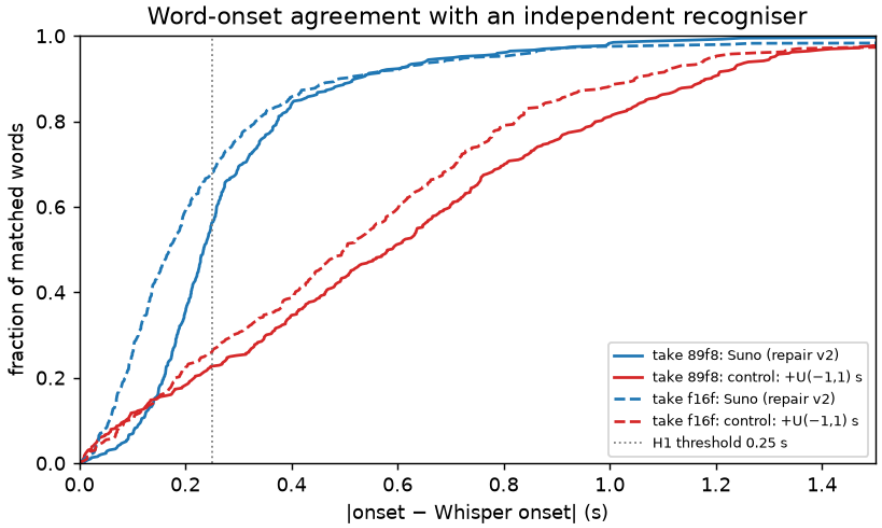


Figure 2: Exp03. Empirical CDFs of word-onset disagreement with the Whisper reference for the timings used in the video (repair v2), and for the random-shift control.

4.4 Render and verification (exp04)

Iteration before the full render. Section-midpoint stills, inspected visually, exposed six layout defects (Table in exp04), all fixed before the full render. Examples: overflowing kinetic words, a lattice-trail oscillation caused by too large a step size, and the ladder colliding with the lyrics.

Final render.

- 7213 frames, H.264 + AAC, 1920×1080, 30/1 fps.
- Container duration 240.45 s against a 240.42 s song, which passed the ≤ 0.5 s criterion.
- The shipped render took 347.3 s (217.6 MB) on a 16-thread desktop with 8 headless-Chrome tabs; an earlier render with v1 timings took 453.6 s.
- The automated check passed.

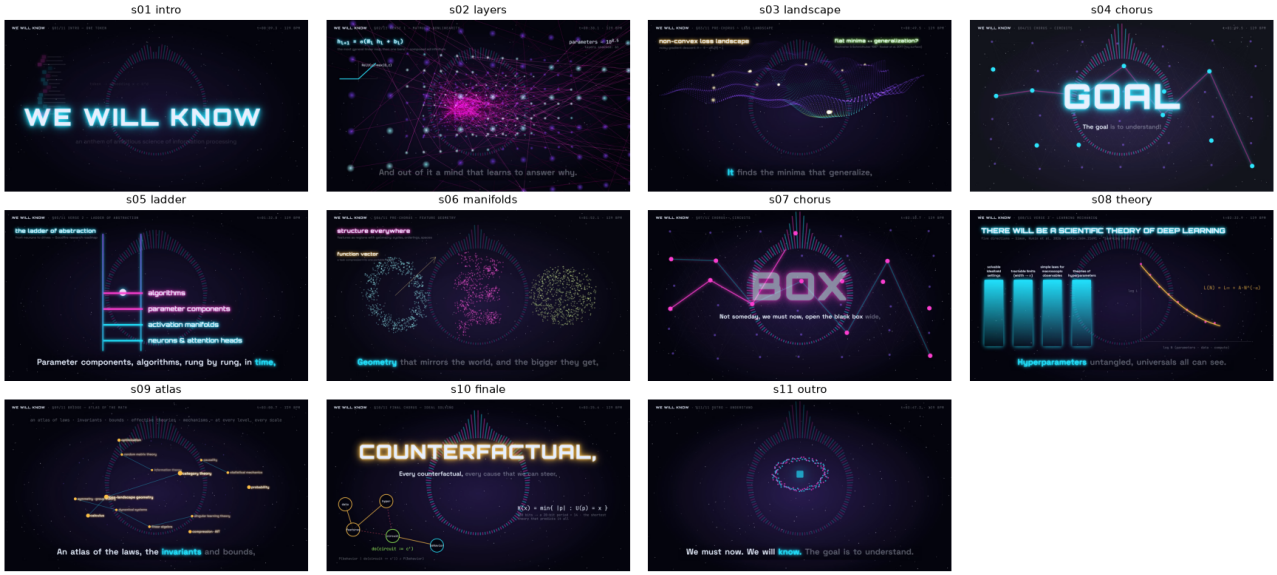


Figure 3: Exp04. One frame from the middle of each section of the final video.

5. Discussion

What the evidence supports.

- The pipeline renders a valid, song-length video deterministically.
- Its beat times are accurate to under one frame on click tracks between 70 and 160 BPM.
- Its word onsets agree with an independent recogniser to about a quarter of a second (median), clearly better than chance. Most of that disagreement is a constant 0.17–0.23 s lag, around which relative timing agrees to about 0.08 s (post-hoc).

What it does not support.

- Any claim about musical quality.
- Any claim about Suno's alignment accuracy *per se*. The 0.235 s median mixes provider and Whisper error, so it bounds a combination of the two.
- The beat accuracy on the actual orchestral mix, which has no ground truth.

The most useful outcome was negative. A reasonable-looking heuristic, written to fix visible artifacts, made things worse on 43 words. Only an independent reference and a pre-stated test revealed this. The general lesson for code-animated music videos is to treat every timing transformation as an intervention and to measure it against something independent, not against the same signal it was designed from.

6. Limitations

- **Sample size.** One generation (n = 1 task, 2 takes) and one song. Nothing here estimates population-level behaviour of Suno or of the provider.
- **Reference quality.** The Whisper reference is noisy: sung vocals over an orchestral mix, and timestamps from cross-attention. Exp03 bounds disagreement; it does not measure error. A stronger test would use a separated vocal stem, a lyrics-aware forced aligner, and manually annotated onsets.
- **Synthetic beat test.** Click tracks lack the onsets and tempo drift of real music, and downbeats rely on a 4/4 heuristic that was not validated.
- **Post-hoc analyses.** Repair v2 and the slope tempo estimator were designed after seeing data. Both are labelled as post-hoc throughout, and no confirmatory claim is made about them.
- **Black-box components.** The visual scenes are schematic; synthetic plot data is labelled. The music model and the provider are black boxes, and hootCer's definition is undocumented.

7. Conclusion

A code-animated music video can be built as an ordinary, testable data pipeline:

- the paid generative step is cached;
- the timing components are validated against references fixed in advance;
- the render is deterministic and checked automatically.

Doing so turned up a quantization bug, a half-tempo failure mode, and a harmful repair heuristic. All three would have been invisible in the finished video's description, and two of them visible on screen. Code, data, logs-derived reports and the video are available at https://github.com/BurnyCoder/ambitious-science-of-information-processing-anthem.

References

- Bain, M., Huh, J., Han, T., & Zisserman, A. (2023). WhisperX: Time-accurate speech transcription of long-form audio. *INTERSPEECH 2023*. arXiv:2303.00747.
- Ellis, D. P. W. (2007). Beat tracking by dynamic programming. *Journal of New Music Research*, 36(1), 51–60.
- Engels, J., Michaud, E. J., Liao, I., Gurnee, W., & Tegmark, M. (2024). Not all language model features are one-dimensionally linear. arXiv:2405.14860.
- Goodfire (n.d.). We can and must solve alignment. https://www.goodfire.com/blog/we-can-and-must-solve-alignment (accessed 18 Oct 2026).
- Gurnee, W., & Tegmark, M. (2023). Language models represent space and time. arXiv:2310.02207.
- Hochreiter, S., & Schmidhuber, J. (1997). Flat minima. *Neural Computation*, 9(1), 1–42.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., & Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. *ICLR 2017*. arXiv:1609.04836.
- Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1), 1–7.
- leogao (2025, December 5). An ambitious vision for interpretability. *LessWrong*. https://www.lesswrong.com/posts/Hy6PX43HGgmfiTaKu/an-ambitious-vision-for-interpretability.
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). librosa: Audio and music signal analysis in Python. *Proc. 14th Python in Science Conference*.
- Pearl, J. (2009). *Causality: Models, reasoning, and inference* (2nd ed.). Cambridge University Press.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *ICML 2023*. arXiv:2212.04356.
- Raffel, C., McFee, B., Humphrey, E. J., Salamon, J., Nieto, O., Liang, D., & Ellis, D. P. W. (2014). mir_eval: A transparent implementation of common MIR metrics. *ISMIR 2014*.
- Simon, J., Kunin, D., Atanasov, A., Boix-Adserà, E., Bordelon, B., Cohen, J., Ghosh, N., Guth, F., Jacot, A., Kamb, M., Karkada, D., Michaud, E. J., Ottlik, B., & Turnbull, J. (2026). There will be a scientific theory of deep learning. arXiv:2604.21691.

- Todd, E., Li, M. L., Sharma, A. S., Mueller, A., Wallace, B. C., & Bau, D. (2023). Function vectors in large language models. arXiv:2310.15213 (ICLR 2024).
- Remotion documentation, https://www.remotion.dev/docs/; sunoapi.org documentation, https://docs.sunoapi.org/ (both accessed 18 Oct 2026).

Appendix A. Reproduction

```
uv sync && (cd video && npm install)
cp .env.example .env          # key only needed for a new generation
uv run anthem                 # cached song -> timeline -> render -> verify
uv run python scripts/validate_beats.py
uv run --group analysis python scripts/validate_alignment.py small.en
uv run --group analysis python scripts/make_figures.py
```

Appendix B. Lyrics structure

The lyrics have 11 sections: Intro, Verse 1, Pre-Chorus, Chorus, Verse 2, Pre-Chorus, Chorus, Verse 3, Bridge, Final Chorus, Outro. The full text is in `content/lyrics.txt`, and the section-to-scene map is in `content/scenes.toml`, described in `docs/scenes.md`.